Machine Learning in Engineering Science

A Colab-first course reader

Chi-Hua Jonny Yu

Fall 2026

How to use this reader

This is the Fall 2026 course reader for *Machine Learning in Engineering Science*. It replaces the 2025 model-by-model textbook structure with a learning sequence that follows the course calendar. The reader is not a catalogue of architectures. Its central question is: *what evidence is sufficient to support an engineering decision?*

Each teaching week combines about 90 minutes of lecture with 90 minutes of practice. For Week 7 these modes are distributed across three independently usable 60-minute modules: sequential data and LSTM, attention, then Transformer architecture and experiments. The website provides the matching notebooks and slide decks. This source keeps the narrative, equations, design rationale, and references in version-controlled LaTeX.

Learning outcomes

By the end of the course, students should be able to:

- formulate an ML problem with a target, features, split, metric, and decision context;
- implement and evaluate a reproducible baseline before seeking a more complex model;
- identify data leakage, overfitting, uncertainty, and failure cases;
- explain when an LLM, retrieval component, tool call, or agent loop is justified; and
- use AI coding assistance while retaining responsibility for tests, evidence, and rollback.

Semester map

Weeks	Course focus	Reader unit and tangible output
1–3	Data workflows and supervised learning	Chapters 1 and 2; a question, a table of features, and a baseline model.
5–6	Evaluation and representation	Chapters 3 and 4; a defensible split, metric, and representation.
7	Sequences, LSTM, attention, and Transformers	Chapters 5 and 6; three one-hour modules with reproducible experiments.
9–10	Reliability and project studio	Chapter 7; a reproducible midterm ML workflow.
11	Midterm assessment	Project demonstration, written examination, and oral examination.
12–14	LLMs, retrieval, tools, and agent design	Chapters 8 to 10; an agent plan with explicit tools and failure cases.
15–17	Vibe coding, tests, and final studio	Chapter 11; an evaluated engineering workflow and final demo.
18	Final assessment	Final project demonstration, written examination, and oral examination.

What is assessed

Labs are active classwork and do not create a weekly graded workload. There are four milestone-based items called *Homework*: an ML problem brief, a reproducible ML workflow, an agent plan, and demo-readiness evidence. The midterm and final each combine a project with written and oral assessment.

A note on AI tools

The course permits free or basic-credit AI tools for drafting code, explanations, and debugging. A generated answer is never sufficient evidence. Submissions must show the model or system used, the prompt or change request when relevant, the tests run, the observed result, and the student's explanation of limits. No project requires a paid API, a dedicated GPU, or a competition ranking.

Contents

How to use this reader i

I Machine Learning Foundations 1

1 Data, Questions, and the ML Workflow 2

1.1 From an engineering question to a learning task . . . 2

1.2 A reproducible first pass . . . 2

1.3 Data contracts . . . 3

2 Supervised Learning: Baselines Before Complexity 4

2.1 Classification and regression . . . 4

2.2 Split before learning from the data . . . 4

2.3 Algorithm choice . . . 5

3 Evaluation, Validation, and Generalization 6

3.1 Metrics carry assumptions . . . 6

3.2 Generalization and the bias–variance trade-off . . . 6

3.3 Validation discipline . . . 7

4 Unsupervised Learning and Representation 8

4.1 Clustering as a hypothesis generator . . . 8

4.2 Representation matters . . . 8

5 Neural Networks and Deep Learning Foundations 10

5.1 A compact mathematical view . . . 10

5.2 Training is an experiment . . . 10

6 Sequential Data, LSTM, Attention, and Transformers 12

6.1 Learning outcomes and evidence . . . 12
6.2 Hour 1: ordered observations and recurrent memory . . . 12
6.2.1 Order, sampling, and the prediction task . . . 12
6.2.2 Chronological splits and available information . . . 13
6.2.3 Baselines and error interpretation . . . 13
6.2.4 RNN state and learning through time . . . 13
6.2.5 The complete LSTM mechanism . . . 14
6.2.6 Implementation, selection, and reproducibility . . . 14
6.3 Hour 2: why attention is useful and how it works . . . 14
6.3.1 The fixed-context bottleneck . . . 14
6.3.2 Queries, keys, values, and shapes . . . 15
6.3.3 An attention calculation with no hidden steps . . . 15
6.3.4 Self-attention, cross-attention, and masks . . . 16
6.3.5 Benefits, costs, and interpretation . . . 16
6.4 Hour 3: assemble and test a Transformer . . . 16
6.4.1 Tokens and position . . . 16
6.4.2 Heads, nonlinear transformations, and residual paths . . . 16
6.4.3 Objective, baseline, and generation . . . 17
6.4.4 Behavioral checks and limits . . . 17
6.5 Completion and explain-back . . . 17
6.6 Primary sources . . . 18

7 Reliability, Error Analysis, and the Midterm Workflow 19
7.1 Week 9: the evidence contract . . . 19
7.1.1 A result is conditional . . . 19
7.2 Residuals, metrics, and mixtures . . . 20
7.2.1 A complete subgroup calculation . . . 20
7.2.2 An outlier worked example . . . 20
7.3 Why the known generator is useful . . . 21
7.3.1 Split, select, then evaluate . . . 21
7.3.2 What a residual shape explains . . . 21
7.4 Two kinds of change . . . 21
7.4.1 Covariate shift . . . 21
7.4.2 An analytic stress test is not the fitted experiment . . . 22

7.4.3 Concept shift can hide from input checks . . . 22
7.5 Interpretability as a checked equation . . . 23
7.6 Uncertainty and evidence identity . . . 23
7.6.1 The bootstrap is conditional . . . 23
7.6.2 Freeze and replay . . . 24
7.7 Eight core exercises and two optional challenges . . . 24
7.7.1 The result card and export . . . 25
7.8 Reference execution and its limits . . . 25
7.9 Summary and next dependency . . . 25
7.10 Further reading . . . 26
7.11 Week 10: Reproducible ML Workflow Studio . . . 26
7.11.1 Computational replay has a bounded meaning . . . 26
7.12 Stable identity, not a seed alone . . . 26
7.13 The packaged regression fixture and its limits . . . 27
7.13.1 An executable interface . . . 27
7.14 Baseline, selection and held-out evidence . . . 27
7.15 A checkable function explanation . . . 28
7.16 What belongs in the handover . . . 29
7.16.1 Integrity is not authentication . . . 29
7.17 Replay outside notebook memory . . . 30
7.18 Test deliberate failures on copies . . . 30
7.19 Eight core exercises and two optional challenges . . . 30
7.19.1 Student responses and evidence export . . . 32
7.20 Instructor execution and limits . . . 32
7.21 Summary and examination bridge . . . 33
7.22 Primary-source reading . . . 33
7.23 Weeks 10–11: workflow and examination preparation . . . 33

II Language Models, Agents, and Engineering Workflows 34

8 Transformers and Large Language Models 35

8.1 Attention and next-token prediction . . . 35
8.2 Use a bounded role . . . 35

9 Retrieval, Tools, and Structured Knowledge **37**
9.1 Retrieval-augmented generation . . . 37
9.2 Tools are software interfaces . . . 37

10 Agent Design for Engineering Workflows **39**
10.1 A minimal agent loop . . . 39
10.2 Agent plan Homework . . . 39
10.3 Current agent engineering patterns . . . 40

11 Testing, Vibe Coding, and the Final Project **41**
11.1 Vibe coding with boundaries . . . 41
11.2 Evaluation before demonstration . . . 41
11.3 Final assessment . . . 42
11.4 Risk-aware evaluation . . . 42

A Colab Conventions and Homework Milestones **43**
A.1 Colab conventions . . . 43
A.2 Homework milestones . . . 43
A.3 Submission rule . . . 44
A.4 Runtime-aware Lab design . . . 44

B Extension Studios: Deep Learning Architectures **45**
B.1 Convolutional neural networks . . . 45
B.2 Sequential models . . . 45
B.3 Generative models . . . 45

Part I

Machine Learning Foundations

Chapter 1

Data, Questions, and the ML Workflow

Weeks 1–2. A useful ML system begins before model selection. It begins with a decision, a measurable target, a representation of the available evidence, and a procedure that can be repeated. This chapter establishes the working vocabulary used throughout the course.

Learning outcomes

- Distinguish a decision problem from an ML task.
- Identify an observation, feature vector x, target y, and prediction $\hat{y}$.
- Create a small, inspectable data workflow in Python.
- State what a plot or summary statistic does *not* establish.

1.1 From an engineering question to a learning task

An engineering question such as "Which process condition is likely to produce a defect?" must be made operational. Define the unit of observation, the time at which the prediction is made, the target, the available features, and the decision that follows. A classification task predicts a discrete label; a regression task predicts a continuous value. Neither label is meaningful without a stated measurement process and error cost.

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ denote observed examples. A learning algorithm selects a function f_θ so that $\hat{y} = f_\theta(x)$ is useful on data not used to choose θ. The course treats this as an evidence problem: the data split and evaluation protocol are part of the model, not afterthoughts [8].

1.2 A reproducible first pass

Before fitting a model, inspect shape, column meaning, units, missingness, and plausible ranges. The following pattern is deliberately small enough to read line by line.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(2026)
df = pd.DataFrame({
    "temperature_c": rng.normal(72, 5, 40),
    "pressure_kpa": rng.normal(210, 18, 40),
})
df["defect_flag"] = (
    (df["temperature_c"] > 75) & (df["pressure_kpa"] > 220)
).astype(int)

print(df.shape)
print(df.describe())
plt.scatter(df["temperature_c"], df["pressure_kpa"],
            c=df["defect_flag"], cmap="coolwarm")
plt.xlabel("Temperature (C)")
plt.ylabel("Pressure (kPa)")
plt.show()
```

A coloured scatter plot can reveal a possible association. It does not prove causality, establish a production threshold, or show that a future batch will behave similarly. The lab asks students to write both the supported observation and a limitation.

Colab Lab

Week 1: Open in Google Colab. Inspect a measurement table, compute descriptive statistics, create one figure, and write an evidence statement. The target is not yet a high-scoring model; it is a clear workflow that another person can reproduce.

1.3 Data contracts

A data contract records the feature names, units, allowable values, target definition, split rule, and known missingness. It is lightweight documentation with a strong engineering benefit: it prevents silent changes from being mistaken for model improvement. For every course notebook, include a first cell that reports package versions, a random seed, data provenance, and the shape of the loaded table.

Checkpoint

For a proposed predictor of fatigue life, name: (1) one observation, (2) three candidate features, (3) the target and its unit, (4) a decision that uses the prediction, and (5) one feature that would be unavailable at prediction time. Explain why the last feature must be excluded.

Chapter 2

Supervised Learning: Baselines Before Complexity

Weeks 2–3. Supervised learning uses labelled examples to learn a mapping from features to a target. The practical discipline is to establish a transparent baseline, then compare alternatives under the same split and metric.

Learning outcomes

- Separate feature engineering from target definition.
- Construct train and test partitions without leakage.
- Fit a baseline regression or classification pipeline.
- Interpret a confusion matrix and a residual plot in context.

2.1 Classification and regression

For binary classification, the model estimates a score or probability $p_\theta(y = 1 \mid x)$. A threshold turns the score into an action. The threshold should follow the cost of false positives and false negatives, not a default convention. For regression, a common starting loss is mean squared error,

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2,$$

but an error in megapascals, minutes, or dollars must be interpreted in its application context.

2.2 Split before learning from the data

Data leakage occurs when information from the test set changes feature construction, scaling, imputation, model selection, or threshold choice. A pipeline makes the correct order explicit: fit transformations on training data only, then apply the learned transformations to held-out data.

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

wine = load_wine(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    wine.data, wine.target, test_size=0.25,
    random_state=2026, stratify=wine.target,
)
model = Pipeline([
    ("scale", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=2000)),
])
model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))
```

A logistic-regression pipeline is not "simple" in the sense of being disposable. It establishes a reproducible reference point. A more complex model earns its place only when the evaluation protocol demonstrates an improvement relevant to the decision.

2.3 Algorithm choice

Decision trees expose rule-like splits; nearest-neighbour methods compare local examples; support vector machines seek a separating margin; ensembles combine diverse predictors. These methods remain useful when the dataset is structured, moderate in size, and the evaluation contract is clear [1, 14]. The course uses them to develop comparison discipline, rather than to encourage architecture shopping.

Colab Lab

Week 2: Open in Google Colab. Build a leakage-safe baseline classifier, report accuracy and a confusion matrix, then change one decision and document its effect. Your explanation is more important than the numerical score alone.

Checkpoint

A feature is calculated using the mean of every row in the dataset before the train/test split. Is this leakage? Describe the corrected order of operations and explain why it matters even if the measured score changes only slightly.

Chapter 3

Evaluation, Validation, and Generalization

Week 5. A score is evidence only when it was produced by a protocol that matches the intended use. This chapter makes error analysis, validation, and model selection explicit.

Learning outcomes

- Select metrics that expose the errors that matter.
- Explain underfitting, overfitting, and the bias–variance trade-off.
- Use cross-validation without contaminating the final test set.
- Turn model errors into a next experimental question.

3.1 Metrics carry assumptions

Accuracy is the fraction of correct predictions, but it can hide failures when classes are imbalanced. Precision asks whether positive predictions are credible; recall asks whether actual positives are found. For a binary classifier,

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Choose the metric before inspecting the final score. A defect-screening model might favour recall, while an expensive destructive test may require higher precision.

3.2 Generalization and the bias–variance trade-off

Training performance can improve while test performance degrades. A model with too little capacity underfits; a model tuned too aggressively to a small training set overfits. The remedy is not always a larger model. Better measurements, fewer unstable features, regularization, or a clearer task definition may be more valuable.

3.3 Validation discipline

Use training data to fit candidate pipelines, validation or cross-validation to select among them, and a final test set for one honest estimate. Record the split seed, preprocessing, metric, and selection rule. Do not repeat test-set evaluation until it becomes an informal validation loop.

Colab Lab

Week 5: compare two pipelines using the same split and metric. Make a plot of training and validation behaviour, then write a short error analysis: which cases fail, what shared property might explain the failure, and what additional data would reduce uncertainty?

Checkpoint

A classifier has 98% accuracy but identifies only 2 of 40 safety-critical failures. Which additional metric would you inspect first, and what decision trade-off does it reveal?

Chapter 4

Unsupervised Learning and Representation

Week 6. Unsupervised methods organize data without a labelled target. They are useful for exploration, anomaly investigation, compression, and finding candidate subpopulations. They do not automatically create ground truth.

Learning outcomes

- Explain the difference between clustering and classification.
- Standardize features appropriately before distance-based analysis.
- Use a low-dimensional view as an exploratory tool rather than a final conclusion.
- State how a discovered cluster would be checked against domain knowledge.

4.1 Clustering as a hypothesis generator

Methods such as k-means partition observations by geometric similarity. The algorithm returns groups even when no meaningful groups exist. Therefore a cluster label should be treated as a hypothesis to investigate with process knowledge, additional measurements, and sensitivity analysis.

4.2 Representation matters

Distances depend on feature scale and representation. A temperature difference of 10 degrees and a pressure difference of 10 kPa are not inherently comparable. Document scaling, missing-value handling, and the variables intentionally excluded from the analysis.

Colab Lab

Week 6: standardize a multivariate dataset, compare two values of k, and inspect the stability of the result. Identify one observation that would require domain review rather than automatic labelling.

Chapter 5

Neural Networks and Deep Learning Foundations

Week 7. Deep learning learns representations through composed differentiable functions. The course focuses on the training loop, loss surface, and diagnostic signals needed to use a neural network responsibly.

Learning outcomes

- Describe a forward pass, loss, gradient, and parameter update.
- Relate activation functions and network capacity to optimization behaviour.
- Read a training curve for divergence, underfitting, and overfitting.
- Compare a neural baseline with a classical baseline under the same protocol.

5.1 A compact mathematical view

For one layer, $h = \phi(Wx + b)$, where W and b are learned parameters and ϕ is a nonlinear activation. A loss $\mathcal{L}(y, \hat{y})$ measures discrepancy. Backpropagation applies the chain rule to compute $\nabla_\theta \mathcal{L}$ efficiently [12]. Gradient-based optimization then updates parameters, for example $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$.

5.2 Training is an experiment

Fix seeds where possible, retain the split, log loss and metric traces, and compare a neural network to a simpler baseline. A training curve is not a certificate of generalization. It is an instrument that helps identify whether the next action is to inspect data, change capacity, regularize, or stop.

Colab Lab

Week 7: train a small PyTorch network on a compact dataset using CPU or optional Colab acceleration. Keep the network deliberately small. Compare it to a classical baseline and write what the extra capacity changed.

Chapter 6

Sequential Data, LSTM, Attention, and Transformers

Week 7: Monday 19 October 2026, 14:00–17:00.

The three one-hour modules follow one question: how can a learned model use earlier information to make a defensible prediction? Hour 1 develops recurrent memory, Hour 2 develops query-dependent retrieval, and Hour 3 assembles and tests a causal Transformer. About 90 minutes of explanation and worked reasoning are combined with 90 minutes of prediction, guided computation, experimentation, and explain-back across the three hours.

6.1 Learning outcomes and evidence

Students should be able to define a forecast origin and horizon; construct windows without future leakage; trace every LSTM gate; compute attention scores, weights, and a context vector; distinguish self-attention from cross-attention; explain positional information, multi-head attention, residuals, and feed-forward layers; and test a decoder's causal behavior. The three independent CPU notebooks compare an LSTM against baselines, verify attention arithmetic, and train and inspect a small Transformer. All data are synthetic or self-contained. No paid API or GPU is required. These are class exercises, not additional weekly graded assignments.

6.2 Hour 1: ordered observations and recurrent memory

6.2.1 Order, sampling, and the prediction task

A sequence is an ordered collection $x_1, \ldots, x_T$ in which order is part of the problem definition. Sensor samples, process events, and language tokens are examples. $(1, 2, 4, 8)$ and $(8, 1, 4, 2)$ have the same multiset and mean but different transitions. A representation that discards order cannot recover those transitions from the values alone. Equally spaced samples and irregular events also require different sampling assumptions. A sequence model does not automatically infer omitted time intervals.

In the example, the forecast origin is immediately before observing x_t. With window length L,

$$X_t = (x_{t-L}, \ldots, x_{t-1}), \qquad y_t = x_t.$$

The Python slice `signal[t-L:t]` excludes index t. Every input index must be strictly less than the target index. This is a one-sample horizon. A fixed-origin multi-step forecast cannot use later actual values that would not yet have been observed.

With B histories, T time steps, and F features at each step, batch-first input has shape (B, T, F). Many-to-one predicts one value per history; many-to-many predicts outputs at multiple positions. Batch size is not sequence length, and an ID is not necessarily a physical feature.

6.2.2 Chronological splits and available information

The synthetic sensor uses 1600 samples, window length 24, training targets before index 960, validation targets 960–1279, and test targets 1280 onward. This produces 936 training, 320 validation, and 320 test windows. Target periods are disjoint. A test window may use earlier observed validation or test sensor history. This is explicitly a rolling one-step protocol, not a forecast of all future values from one initial origin.

Normalization learns statistics, so fit it on training observations only:

$$z_t = (x_t - \mu_{\text{train}})/\sigma_{\text{train}}.$$

Global means can leak future distribution information even without labels. Random splits of heavily overlapping windows can produce nearly identical training and test histories. For multiple devices or people, group separation may be needed in addition to temporal separation.

6.2.3 Baselines and error interpretation

Persistence uses $\hat{x}_t = x_{t-1}$ and has no fitted parameters. Ridge regression uses a regularized linear map of the flattened history window. Select regularization on validation, then compare the same test targets:

$$\text{MAE} = \frac{1}{N}\sum_n |\hat{y}_n - y_n|, \quad \text{RMSE} = \sqrt{\frac{1}{N}\sum_n (\hat{y}_n - y_n)^2}.$$

Report original target units and inspect residuals against time or operating range. RMSE emphasizes large errors. Neither score alone reveals every failure. A linear model can outperform an LSTM on a small, structured signal; report the observed comparison rather than choosing the more complex name.

6.2.4 RNN state and learning through time

A basic recurrent network reuses one update at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

Unrolling displays repeated applications of shared parameters. The state is a learned summary, not necessarily an interpretable physical variable. Backpropagation through time includes products of

Jacobians:

$$\frac{\partial h_t}{\partial h_{t-k}} = J_t J_{t-1} \cdots J_{t-k+1}.$$

Repeated contractions may shrink gradients, and repeated expansions may amplify them. A scalar example 0.9^{100} illustrates one contraction mechanism but not a complete nonlinear Jacobian product. Gradient clipping limits large gradient magnitude; it does not itself solve vanishing gradients.

6.2.5 The complete LSTM mechanism

An LSTM separates cell memory c_t from hidden output h_t. With $u_t = [h_{t-1}; x_t]$, a standard cell uses

$$f_t = \sigma(W_f u_t + b_f), \qquad i_t = \sigma(W_i u_t + b_i), \tag{6.1}$$

$$g_t = \tanh(W_g u_t + b_g), \qquad o_t = \sigma(W_o u_t + b_o), \tag{6.2}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{6.3}$$

Here $\sigma(z) = 1/(1 + e^{-z})$ and $\odot$ is elementwise multiplication. Each gate is a vector of coefficients, generally one per hidden dimension. The forget gate retains old cell content; the input gate scales a signed candidate write; the output gate exposes a filtered view of the updated cell. The additive memory path can improve gradient flow but does not guarantee unlimited retention or eliminate every optimization difficulty. The direct cell-state path contains products of forget gates when their other dependencies are held fixed; the full derivative is more complicated.

Choose illustrative values $c_{t-1} = 0.8$, $f_t = 0.9$, $i_t = 0.2$, $g_t = -0.5$, and $o_t = 0.7$. Then

$$c_t = 0.72 - 0.10 = 0.62, \qquad h_t = 0.7 \tanh(0.62) \approx 0.386.$$

These are hand-chosen teaching values, not training outputs. If $f_t = 1$, new content can still change the cell. Changing o_t changes hidden output without changing this cell update. Predict those effects before using sliders.

6.2.6 Implementation, selection, and reproducibility

With `batch_first=True`, input/output layouts are (B, T, F) and (B, T, H). Ordinary non-projected hidden and cell states retain layout (layers $\times$ directions, B, H). PyTorch concatenates gate chunks in input, forget, candidate, output order. The notebook reconstructs a learned cell and compares it numerically with `nn.LSTM`. Training updates use training data, validation selects a checkpoint, and test is evaluated afterward. Save raw data identity, seed, versions, window, split boundaries, normalization, model state, baselines, and measured errors. A seed alone does not guarantee bitwise agreement across environments.

6.3 Hour 2: why attention is useful and how it works

6.3.1 The fixed-context bottleneck

A final-state encoder compresses a source sequence into one fixed-width context. Different output steps may need different earlier details. Attention makes encoder states addressable and forms a

context for each query. It does not guarantee correct retrieval or unlimited context. Attention can be combined with recurrence: Bahdanau and colleagues used recurrent sequence-to-sequence modeling in a 2014 preprint presented in 2015. The Transformer architecture came later. Distinguish a retrieval mechanism from the entire architecture that uses it.

6.3.2 Queries, keys, values, and shapes

Queries encode a retrieval need. Keys encode matching information, while the corresponding value rows carry content to combine. For N_q queries, N_k keys, key width d_k, and value width d_v,

$$Q \in \mathbb{R}^{N_q \times d_k}, \quad K \in \mathbb{R}^{N_k \times d_k}, \quad V \in \mathbb{R}^{N_k \times d_v}.$$

Values need not equal keys or have their width. A natural-language question is not the same object as the query vector produced by a learned projection.

Additive attention can use

$$e_{ij} = v_a^\top \tanh(W_q q_i + W_k k_j + b).$$

Scaled dot-product attention uses

$$S = QK^\top/\sqrt{d_k}, \quad A_{ij} = \frac{\exp(S_{ij})}{\sum_m \exp(S_{im})}, \quad C = AV.$$

Scores compare, row-wise softmax normalizes, and value multiplication retrieves. Rows are query destinations and columns are key sources. The context has shape (N_q, d_v).

Under independent zero-mean unit-variance coordinate assumptions, the raw dot-product variance grows with d_k. Scaling by $\sqrt{d_k}$ controls that growth; it is not cosine normalization. Stable softmax subtracts the row maximum before exponentiation without changing normalized probabilities. Before later projections and residuals, the weighted context lies within the convex hull of the available values.

6.3.3 An attention calculation with no hidden steps

Choose teaching vectors

$$Q = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 10 & 0 \\ 0 & 10 \\ 10 & 10 \end{bmatrix}.$$

Raw dot products are $(1, 0, 1)$ and scaled scores are approximately $(0.7071, 0, 0.7071)$. Softmax gives weights $(0.4011, 0.1978, 0.4011)$. The zero raw score does not give zero weight, because $\exp(0) = 1$. Combining corresponding value coordinates gives

$$C_1 = 0.4011(10) + 0.1978(0) + 0.4011(10) \approx 8.0222,$$
$$C_2 = 0.4011(0) + 0.1978(10) + 0.4011(10) \approx 5.9889.$$

Use full precision in code. Change only the query and predict the new ranking. An optional positive temperature divides the scores: below one sharpens weights, above one softens them. The standard example uses one.

6.3.4 Self-attention, cross-attention, and masks

Self-attention projects one sequence X into $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. Cross-attention derives queries from one stream and keys/values from another. An encoder-decoder model uses decoder queries with encoder memory. A decoder-only model normally has no separate source encoder.

In causal next-token training, input position i predicts token $i+1$, so it may access itself and earlier inputs: $j \leq i$. Set future scores to $-\infty$ before softmax, yielding zero weights. Padding masks hide nonexistent tokens instead. Combine permissions and exclude padded query outputs from loss. An all-masked row has no valid normalization; prevent it or specify a deliberate special case. Mask conventions differ across APIs. PyTorch scaled-dot-product attention uses Boolean True for allowed entries, whereas some other attention calls use True for blocked entries.

6.3.5 Benefits, costs, and interpretation

One dense attention layer provides a direct query-to-key computational path to a distant position. Known query operations can run in parallel. Dense score interactions involve T^2 pairs per head: doubling context length quadruples pair count. Exact fused implementations can reduce intermediate memory without changing dense pair-count scaling.

Attention weights describe routing within the computation; they do not prove causal importance. Keep Q and K fixed and change V: weights stay identical but the result can change. Removing and renormalizing a source is a different intervention. Explain what a particular experiment supports rather than interpreting an attractive heatmap as a complete causal explanation.

6.4 Hour 3: assemble and test a Transformer

6.4.1 Tokens and position

Token IDs index learned vectors; their numeric magnitude has no inherent semantic distance. The teaching model has eight symbolic tokens, width 32, four heads, and context length 16. Vocabulary size and embedding width differ. Unmasked attention without positional information is permutation-equivariant: permuting input rows permutes outputs correspondingly. Order must therefore be represented deliberately. Our model adds learned absolute position vectors. The original Transformer used sinusoidal encodings; other position mechanisms also exist. A causal mask already introduces order constraints, so do not apply the unmasked equivariance claim unchanged to a masked model.

6.4.2 Heads, nonlinear transformations, and residual paths

Each head uses learned projections and its own routing weights. Concatenate head outputs and apply a learned output projection. At fixed total width D, H heads have width D/H in this implementation. More heads do not automatically multiply the total width, and heads have no assigned semantic roles. Attention mixes positions, while a shared tokenwise MLP transforms features independently at each position. The notebook expands to $4D$, applies GELU, and projects back to D.

Residual addition requires matching dimensions and preserves an identity route. The teaching decoder uses pre-normalization:

$$u = x + \mathrm{Attention}(\mathrm{LN}(x)),$$
$$y = u + \mathrm{FFN}(\mathrm{LN}(u)).$$

The 2017 paper used a full encoder-decoder with post-normalization and ReLU. Our one-block decoder is an explicitly identified educational variant, not a verbatim copy of that architecture. Its causal self-attention has no encoder-decoder cross-attention sublayer.

6.4.3 Objective, baseline, and generation

Shift targets one token ahead. Logits have shape (B, T, V), and cross-entropy minimizes

$$\mathcal{L} = -\frac{1}{BT} \sum_{b,t} \log p_\theta(y_{b,t} \mid x_{b,\le t}).$$

Pass logits directly to `nn.CrossEntropyLoss`. Training computes many known positions in parallel under the causal mask. During generation, the model predicts one unknown token, appends it, and repeats. Parallel training does not imply arbitrary autoregressive generation is parallel.

Separate synthetic streams use seeds 2026, 2027, and 2028 but share one motif with small random perturbations. This is generalization to new realizations of that process, not transfer to a different engineering domain. A training-only smoothed bigram model is the reference. Uniform prediction over eight tokens has NLL $\log 8 \approx 2.079$ nats/token. Select a checkpoint by validation NLL; evaluate test afterward.

6.4.4 Behavioral checks and limits

Plot an actual trained attention head with query rows and key columns. Upper-triangle zeros are mask constraints, not learned discoveries. Change tokens only at positions 8 onward and compare logits at positions 0–7. They should remain unchanged within the declared tolerance. This tests one concrete causal behavior, not every deployment property. Separately test permutation equivariance of the unmasked operation. Removing positions from an already trained decoder is an inference intervention, not a fair separately trained architecture comparison.

The model generates symbolic states, not evidence-backed natural language. A token generator is only one possible component of an agent. An agent also needs a bounded objective, state handling, validated tools, permissions, stopping rules, and evidence review. Generation alone does not provide factual verification or controlled execution.

6.5 Completion and explain-back

H1 must produce a baseline table, residual plot, a reconstructed learned cell update, and evidence JSON. H2 must produce scores, weights, context, mask assertions, and an interpretation-limiting intervention. H3 must produce a trained-model comparison, head plot, future-token check, and configuration record. Predict the effect of one change before executing, keep the original evidence, and finish with a claim, evidence, limitation, and a test that could challenge the claim.

6.6 Primary sources

- Hochreiter and Schmidhuber (1997), *Long Short-Term Memory*: https://doi.org/10.1162/neco.1997.9.8.1735.
- Bahdanau, Cho, and Bengio (2014/2015), recurrent attention: https://arxiv.org/abs/1409.0473.
- Vaswani et al. (2017), *Attention Is All You Need*: https://arxiv.org/abs/1706.03762.
- Dive into Deep Learning: https://d2l.ai/chapter_recurrent-modern/lstm.html and https://d2l.ai/chapter_attention-mechanisms-and-transformers/transformer.html.
- Executable course notebooks: https://github.com/CHjonnyYu/MLES-Fall-2026.

Chapter 7

Reliability, Error Analysis, and the Midterm Workflow

Weeks 9–11. Reliability connects model results to the populations, operating conditions, inputs and risks that produced them.

Learning outcomes

- Segment error by operating condition and retain sample counts.
- Distinguish a fitted function explanation from a causal claim.
- Preserve exact data, configuration and an executable evidence trail.
- Explain choices and failure cases in writing and orally.

7.1 Week 9: the evidence contract

Meeting: Monday, 2 November 2026, 14:00–17:00. This week allocates 60 minutes to explanation and 120 minutes to practice. H1 contains 45 minutes of explanation and 15 minutes of hand calculation; H2 contains 15 minutes of explanation and 45 minutes of guided computing; H3 is a 60-minute lab. There are eight core exercises and two optional challenges. They are in-class formative practice, not new weekly graded homework.

By the end, students should distinguish aggregate and subgroup error, diagnose a missing feature term, distinguish input shift from target-mechanism shift, interpret a standardized model in original feature coordinates, and replay a result from an exact CSV. Prerequisites are regression, held-out evaluation, MAE, and Python arrays.

7.1.1 A result is conditional

Let f be a fixed fitted predictor and L a declared loss. Population risk is

$$R_P(f) = \mathbb{E}_{(X,Y)\sim P}[L(Y, f(X))].$$

A finite test average estimates this risk under sampling assumptions; it is not the population expectation itself. A score on population P does not automatically establish the same score on another population Q. Name the population, split, units, loss, group counts and model version whenever making a performance claim. A successful software execution is not equivalent to validation of a deployed engineering system.

7.2 Residuals, metrics, and mixtures

We use the sign convention $e_i = \widehat{y}_i - y_i$. Positive residuals mean overprediction, negative residuals mean underprediction. Some references reverse this convention; the plots and interpretation must remain consistent.

$$\mathrm{MAE} = \frac{1}{n}\sum_i |e_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i e_i^2}.$$

Both metrics have response units. MSE has squared response units. RMSE is at least MAE for the same equally weighted rows, but neither metric is universally the correct decision criterion. Declare the loss and the costs it represents before selecting a winner.

7.2.1 A complete subgroup calculation

For disjoint exhaustive groups g with n_g observations and $w_g = n_g/n$,

$$\mathrm{MAE} = \sum_g w_g \,\mathrm{MAE}_g, \qquad \mathrm{RMSE} = \sqrt{\sum_g w_g \,\mathrm{RMSE}_g^2}.$$

The first identity follows by partitioning the absolute-error sum; the second follows by partitioning the squared-error sum. Directly averaging subgroup RMSEs is generally wrong.

Consider 90 observations with absolute error 1 and 10 with absolute error 10. Total absolute error is $90 + 100 = 190$; overall MAE is 1.9. The subgroup MAEs are 1 and 10. Changing only the mixture to 75/25 gives $(75 + 250)/100 = 3.25$. Neither subgroup improved or deteriorated. An aggregate dashboard can therefore change with population composition.

Report counts alongside all slice metrics. An empty group has no defined empirical mean, not an error of zero. A group of two and a group of two hundred can share the same mean but supply very different amounts of evidence. Repeatedly searching final-test boundaries for a favourable score is exploratory analysis, not independent confirmation.

7.2.2 An outlier worked example

For residuals $[0.1, -0.1, 0.2, -0.2]$, $\sum |e_i| = 0.6$ and $\sum e_i^2 = 0.1$. Thus MAE is 0.15 and RMSE is $\sqrt{0.025} \approx 0.1581$. Appending a fifth residual of 3 changes the sums to 3.6 and 9.1. With $n = 5$, MAE becomes 0.72 and RMSE becomes $\sqrt{1.82} \approx 1.3491$. The large squared contribution explains the stronger response of RMSE. Remember that appending changes the denominator too.

7.3 Why the known generator is useful

The teaching dataset has 500 independent synthetic observations:

$$X_0, X_1, X_2 \overset{\text{iid}}{\sim} \mathcal{N}(0,1), \quad \epsilon \sim \mathcal{N}(0, 0.2^2), \quad Y = 2X_0 + X_1^2 + \epsilon.$$

Inputs are dimensionless synthetic numbers, not measured physical quantities. The response has synthetic response units. X_2 is a negative control. The known relation is stipulated for teaching; we have not discovered a physical law.

Two candidates are declared before final evaluation. The first applies training-fitted standardization and Ridge regression to $[x_0, x_1, x_2]$. The second adds the fixed feature x_1^2, then applies training-fitted standardization and Ridge. Both use $\alpha = 1$ and an SVD solver. The linear candidate cannot exactly represent a squared term. Standardization alone is affine and cannot manufacture that missing basis function.

7.3.1 Split, select, then evaluate

Reserve 350 training rows, 75 validation rows and 75 test rows. Store every row ID and its split. Check all three pairwise intersections are empty and the union contains exactly all 500 rows. Random splitting is appropriate here because the observations are independent synthetic draws. Repeated machines or temporal records require grouped or chronological evaluation instead.

Fit scalers and regression parameters on training rows only. Select the candidate with the lowest validation MAE; exact ties favour the linear candidate. Freeze this choice before opening final test evidence. Report the predeclared comparison of both candidates, but do not change the selected winner based on final-test performance. This protocol retains training-only fits; it does not silently refit on training plus validation.

7.3.2 What a residual shape explains

A simple noise-free linear approximation uses $2x_0 + 1$ in place of $2x_0 + x_1^2$. Its residual is $1 - x_1^2$, giving negative tails. The actual fitted model includes noise, finite-sample coefficients and regularization, so its scatter need not equal this exact curve. In real observational data, a curved residual is a diagnostic clue, not proof of a particular physical cause.

7.4 Two kinds of change

7.4.1 Covariate shift

Under covariate shift, $P(X)$ changes while the stipulated $P(Y \mid X)$ remains fixed. The lab draws a separate 150-row base population and noise vector. For severity $s \in \{1, 1.5, 2.5\}$, copy the base observations, multiply only x_1 by s, and recompute $y = 2x_0 + x_1^2 + \epsilon$ using the same noise. Keep x_0, x_2, fitted parameters and candidate selection unchanged. Pairing conditions makes comparisons less contaminated by new random draws; it does not remove all uncertainty.

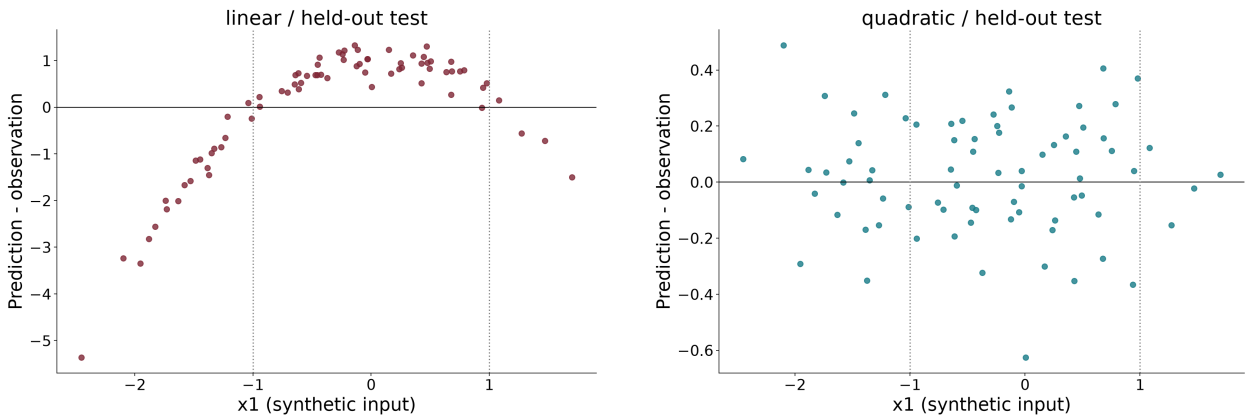


Figure 7.1: Actual synthetic reference test residuals. Sign: prediction minus observation. The panels have independently chosen vertical scales; compare numerical magnitudes, not just occupied plot area.

An exactly represented known function may respond much better than a misspecified linear candidate in this experiment. Do not infer that all distribution shifts necessarily worsen every model, or that a particular finite-sample error ratio is universal.

7.4.2 An analytic stress test is not the fitted experiment

For the fixed toy predictor $\widehat{y} = 0$ and target $y = x^2$, absolute residual equals x^2. For $X \sim \mathcal{N}(0, \sigma^2)$, population MAE is σ^2. A different toy uses 101 endpoint-inclusive, equally spaced points on $[-L, L]$. Its grid MAE is $0.34L^2$, giving 0.34 for $L = 1$ and 3.06 for $L = 3$. The exact ratio of 9 follows from scaling the same normalized grid. The finite-grid mean differs from the continuous uniform expectation $L^2/3$. Neither toy is the fitted Ridge experiment.

7.4.3 Concept shift can hide from input checks

Keep exactly the same test inputs, noise and model predictions, but define changed outcomes $y' = -2x_0 + x_1^2 + \epsilon$. Now the conditional target relation changes while the input rows and histograms remain identical. An input-only monitor cannot detect this constructed change. Comparing predictions with the changed outcomes can expose changed error. In a real system, outcome availability, delay and selection must also be considered.

An input-range flag is not an error label. Observed training min/max describe a finite sample. Gaussian support is unbounded, so being outside the observed range does not imply being outside mathematical support. Changed inputs motivate investigation; outcomes or another justified procedure are needed to assess prediction error.

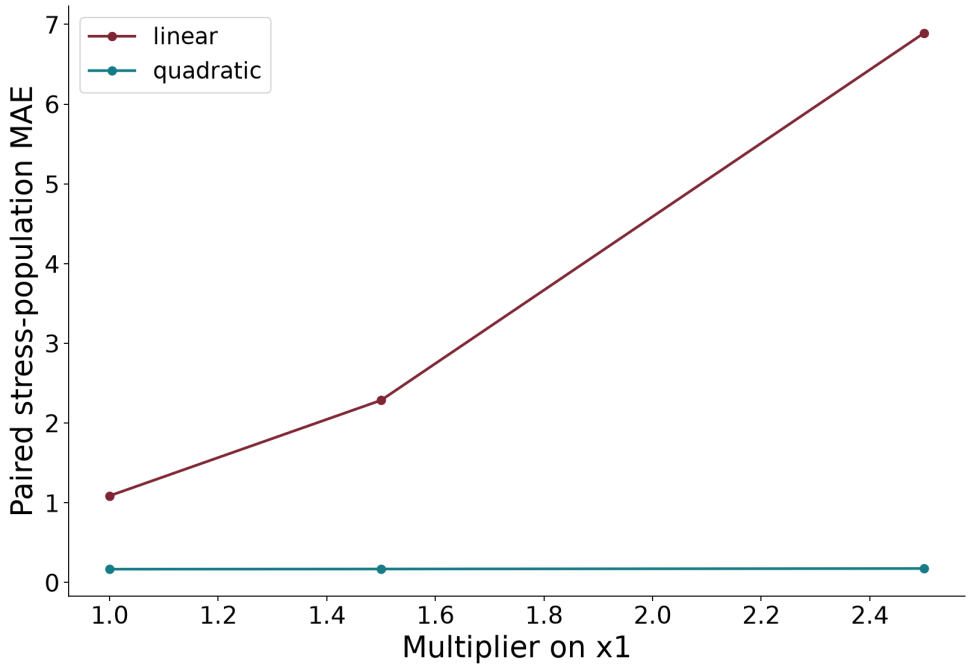


Figure 7.2: Actual errors of frozen fitted candidates on paired synthetic stress populations. These are specified teaching conditions, not external deployment validation.

7.5 Interpretability as a checked equation

For one standardized feature, $z = (x - \mu)/s$ and model term βz give a raw coefficient β/s. For all transformed features,

$$\beta_j^{\text{raw}} = \beta_j / s_j, \qquad b^{\text{raw}} = b - \sum_j \beta_j \mu_j / s_j.$$

Use the scaler fitted after the fixed basis expansion. Reconstruct the complete raw-coordinate prediction and assert agreement with the pipeline. Task 5 perturbs x_0 by 0.5. Its predicted difference is exactly $0.5\beta_{x_0}^{\text{raw}}$ for these models. This describes the fitted function conditional on its representation; it is not evidence of real-world causation.

The optional symmetry check flips x_1. The squared feature is even, but a fitted linear x_1 term can produce $f(-x_1) - f(x_1) = -2\beta_{x_1}^{\text{raw}} x_1$. Noise and regularization do not guarantee that the fitted odd coefficient is exactly zero.

7.6 Uncertainty and evidence identity

7.6.1 The bootstrap is conditional

The notebook resamples absolute test residuals within a slice, computes 300 resampled means, and reports the 2.5th and 97.5th percentiles. This is a descriptive fixed-model interval under an IID-within-slice assumption. It does not cover all training, selection or future-shift uncertainty. For fewer than two observations, no interval is reported. Narrowness is not a safety certificate.

7.6.2 Freeze and replay

Export an exact CSV containing row ID, split, features and target; compute its SHA-256. Retain configuration, package versions, candidate definitions, selected winner, metrics, counts, predictions and original-space coefficients. A hash identifies bytes relative to a recorded digest; it does not validate a scientific claim or labels.

Load the frozen CSV, recover saved training and test membership, clone the already selected candidate, fit on its training rows, and compare its test predictions with the original vector. The notebook uses absolute and relative tolerances of 10^{-10}. Record the actual maximum difference. Agreement under this recorded environment is a reproducibility result, not a guarantee of bitwise identity on every platform.

Task 7 appends a blank line to a copy of the CSV. Its bytes and checksum change while the parser recovers the same 500 nonempty records. The original remains unchanged. This is a concrete counterexample to confusing byte identity with semantic equality or scientific validity.

7.7 Eight core exercises and two optional challenges

Each core exercise requires a prediction written before execution and an explanation afterward. Record the actual observation, an equation or mechanism, and a stopping check. Work in pairs and alternate driver and reviewer. The full exercise notebook includes the walkthrough, so it can be opened on its own.

1. **Mixture calculation (H1):** derive 1.9 and 3.25; show weights and subgroup counts. Stop when both subgroup losses and population change are explicit.
2. **Slice sensitivity (H2):** compare boundaries $|x_1| = 0.5, 1, 1.5$ on frozen test predictions. Report both group counts and MAEs; each pair must sum to 75. Label this exploratory, not model selection.
3. **Grid stress (H1):** derive the factor of 9 before running the 101-point grid. Explain the distinction from a Gaussian population.
4. **Outlier metrics (H1):** show both intermediate sums and denominators before checking MAE/RMSE with code.
5. **Function perturbation (H3):** add 0.5 to x_0 only and verify the raw-coefficient prediction difference within tolerance.
6. **Paired extension (H3):** predeclare scale 2.0, reuse base stress inputs and noise, and compare both actual candidate metrics. Assert parameter snapshots unchanged. Preserve the original reference conditions separately.
7. **Byte identity (H3):** alter a copy by one blank line, verify different hash and identical parsed rows, and confirm the original hash unchanged.
8. **Concept shift (H3):** change the stipulated target coefficient of x_0 from 2 to -2, leaving inputs and predictions fixed. Compare actual errors and explain the input-monitor blind spot.

Optional challenge A checks fitted-function symmetry under $x_1 \mapsto -x_1$. Optional challenge B multiplies served x_0 by 100 without changing intended targets, then decodes by dividing by 100

and verifies restoration of original predictions. These dimensionless synthetic values illustrate an encoding mismatch, not a measured physical unit conversion. Optional work is not required for core completion.

7.7.1 The result card and export

The 19 required response fields comprise eight predictions, eight explanations, one controlled change, one limitation and an AI-use/checking disclosure (or "none"). Missing responses produce status `draft`; nonempty fields produce `ready for peer discussion`, not a correctness grade. Rerun only the final export after editing responses: rerunning initialization would erase them.

Retain both the reference ZIP and the exercise ZIP. The exercise record contains observations and completeness status. Download before closing a free Colab runtime. No paid AI, GPU, external data or API key is required. AI may assist with a hypothesis or code explanation, but every claimed number must come from actual execution. Record material assistance and checks.

7.8 Reference execution and its limits

The accompanying source notebooks were executed from top to bottom in fresh isolated CPU Jupyter kernels on 19 October 2026. This is not a claim of execution in the Google Colab browser. Students must record their own environment and results. Model training, held-out evaluation, 14 figure exports, CSV replay, ZIP integrity, all core tasks, optional branches, and draft/completed export branches were checked locally. Public student notebooks retain blank outputs and responses.

Candidate	Test n	Test MAE	Test RMSE
linear	75	1.031530	1.319474
quadratic	75	0.164319	0.205100

Evidence identity (SHA-256):

`994925f73576edc48f352b4b7300221eda8ed4c117e4f3615382579adf9411ab`

This hash identifies the exact reference CSV. The generated relation and all evaluation populations are synthetic; these outputs do not establish performance on real machines, financial data, or a deployed service.

7.9 Summary and next dependency

Always pair averages with populations and group counts. Use residuals to motivate explicit hypotheses. Distinguish a changed input distribution from a changed target mechanism. Freeze fitted models during stress evaluation, interpret coefficients in the correct coordinates, and replay exact inputs before claiming reproducibility. Week 10 turns this evidence trail into a compact workflow demonstration. Written and oral examination requirements remain unchanged.

7.10 Further reading

- Google Machine Learning Crash Course, monitoring pipelines: Monitoring pipelines documentation.
- scikit-learn, common pitfalls and leakage-safe preprocessing: Common pitfalls documentation.
- scikit-learn, regression evaluation: Regression evaluation documentation.
- Koh et al., WILDS (2021), natural distribution-shift benchmarks: WILDS paper and benchmark description. This is contextual research, not the dataset used in this lab.

7.11 Week 10: Reproducible ML Workflow Studio

Monday, 9 November 2026, 14:00–17:00. The class contains 60 minutes of explanation and 120 minutes of practice. H1 uses 45/15 minutes, H2 uses 15/45, and H3 is a 60-minute studio. Eight core exercises and two optional challenges are formative in-class practice, not new weekly graded homework. The next scheduled meeting is the written and oral midterm examination; this studio does not replace either examination.

The goal is a portable handover: another interpreter receives frozen files, checks their contract, reconstructs a declared model, and compares predictions by stable row identity. A matching screenshot, aggregate score or second notebook variable is not sufficient. Week 9 studied where a model fails; Week 10 asks whether the evidence supporting that analysis can travel.

7.11.1 Computational replay has a bounded meaning

Here, replay means recomputing predictions from the same frozen input records and declared procedure. A new interpreter and working directory remove the notebook's global variables and fitted-object memory. They do not create a new dataset, machine, package environment or physical experiment. Thus numerical agreement is narrower than independent scientific replication. A reproducible model can still be inaccurate or inapplicable.

A two-row counterexample clarifies the distinction between score and prediction identity. Errors $[0, 2]$ and $[2, 0]$ both have MAE 1, but fail on different observations. Compare aligned prediction vectors, not only their mean loss.

7.12 Stable identity, not a seed alone

A seeded permutation selects positions in an ordered array. If the array order changes first, the same positions may identify different observations. Assign source IDs before shuffling and persist them with their measured values. To reconstruct a cohort, build an ID-keyed lookup and iterate over the saved ID sequence. A set comparison verifies membership; a sequence comparison additionally verifies alignment.

The partition contract requires unique row IDs, no duplicates within any split, pairwise disjoint train/validation/test sets, the expected counts, and a union equal to the full frozen dataset.

Disjointness alone does not catch omitted rows. Source row IDs are version-specific dataset identifiers, not globally stable patient identities or predictive features.

7.13 The packaged regression fixture and its limits

The original Week 10 material used scikit-learn's bundled diabetes regression dataset. We retain its 442 observations and 10 numeric inputs but call `load_diabetes(scaled=False)`. The default loader returns globally centered/scaled inputs; raw loading makes the boundary of training-only fitted preprocessing explicit. We do not assign medical interpretations to individual features.

Use boundary: this is a statistical workflow exercise, not diagnosis, treatment guidance, causal discovery or external clinical validation. A random-row teaching split cannot establish future-time, new-site or clinical deployment validity. Dataset documentation also cautions that some feature meanings are not fully explicit.

One seeded permutation reserves 265 training, 88 validation and 89 final-test rows. Store exact floats, feature names, row IDs and split lists. The notebook checks finite numeric values, shape, counts and ID membership before fitting.

7.13.1 An executable interface

The CSV header must exactly match `row_id`, the ten declared feature names in canonical order, and `target`. Duplicate IDs, non-finite values and reordered columns are rejected. Strict rejection is appropriate for a positional model interface; a separately declared and tested name adapter may create a valid canonical interface. Silently guessing feature order is not an acceptable repair.

These checks establish well-formedness relative to a contract, not scientific validity. A finite label can still be wrong, a unique ID can be attached to a wrong observation, and a correctly shaped dataset can be biased.

7.14 Baseline, selection and held-out evidence

Declare four candidates before final evaluation: a training-target median baseline and training-scaled Ridge models with $\alpha \in \{1, 20, 100\}$. A median minimizes $\sum_i |y_i - c|$ over constant predictions c. For an even sample size, any value between the two middle order statistics is a minimizer. That training property does not guarantee the best held-out score.

For the standardized design matrix Z, scikit-learn Ridge uses a residual-sum-of-squares objective plus an L2 penalty:

$$\min_{b,\beta} \sum_{i \in \text{train}} (y_i - b - z_i^{\mathsf{T}}\beta)^2 + \alpha \sum_j \beta_j^2.$$

The intercept is not penalized in this formulation. Standardization and all coefficients are fitted on training data only. Stronger regularization changes the fitted function and coefficient norm; validation MAE need not improve monotonically. The training objective and selection metric need not be identical.

Choose the lowest validation MAE, breaking exact ties by the declared order: median, Ridge 1, Ridge 20, Ridge 100. Freeze the selected candidate before opening final-test results. The replay

program refits this already selected candidate on training rows; it does not redo selection or refit on training plus validation.

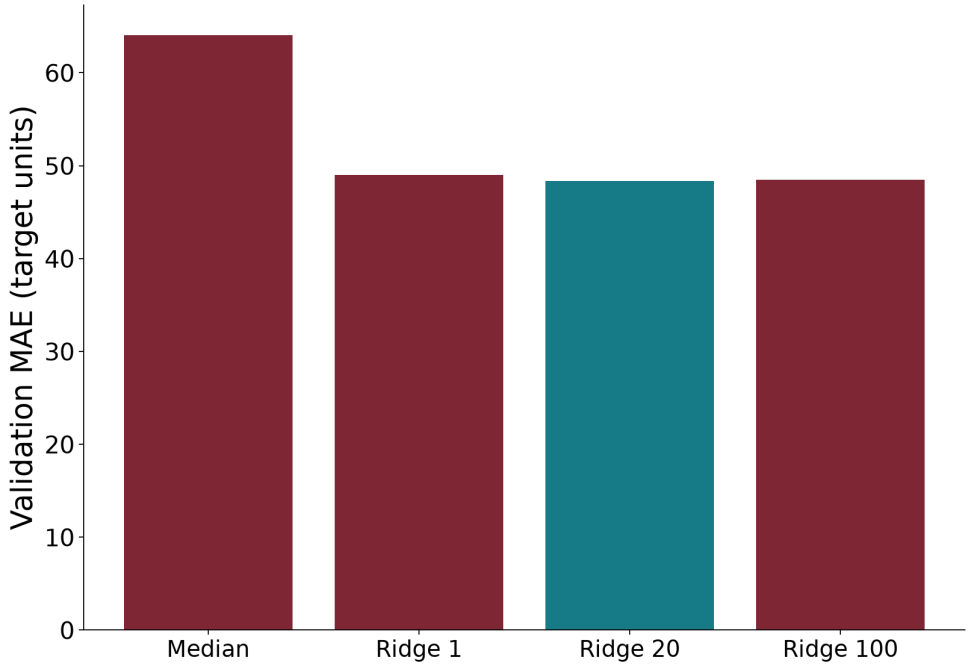


Figure 7.3: Actual validation comparison on 88 rows. The reference run selects Ridge 20. Students retain their own actual environment and outputs.

Report the selected candidate and the predeclared median baseline on the same 89 final-test observations. The residual convention is prediction minus observation; positive means overprediction. Inspect one actual large-residual source ID and its observed/predicted values. Do not infer a clinical mechanism from that row or tune alpha after inspecting it.

Candidate	Test n	MAE	RMSE
median	89	65.280899	79.418957
ridge-20	89	48.325756	59.411283

7.15 A checkable function explanation

For $z_j = (x_j - \mu_j)/s_j$, standardized coefficient β_j gives

$$\beta_j^{\text{raw}} = \frac{\beta_j}{s_j}, \qquad b^{\text{raw}} = b - \sum_j \frac{\beta_j \mu_j}{s_j}, \qquad f(x) = b^{\text{raw}} + \sum_j \beta_j^{\text{raw}} x_j.$$

Reconstruct predictions and compare them to the pipeline. Increasing one input by $0.1 s_j$ changes the fitted prediction by $0.1\beta_j$, with other coordinates fixed. This is an exact property of the additive fitted function, up to numerical tolerance. It need not correspond to a feasible causal or clinical intervention, especially with correlated features.

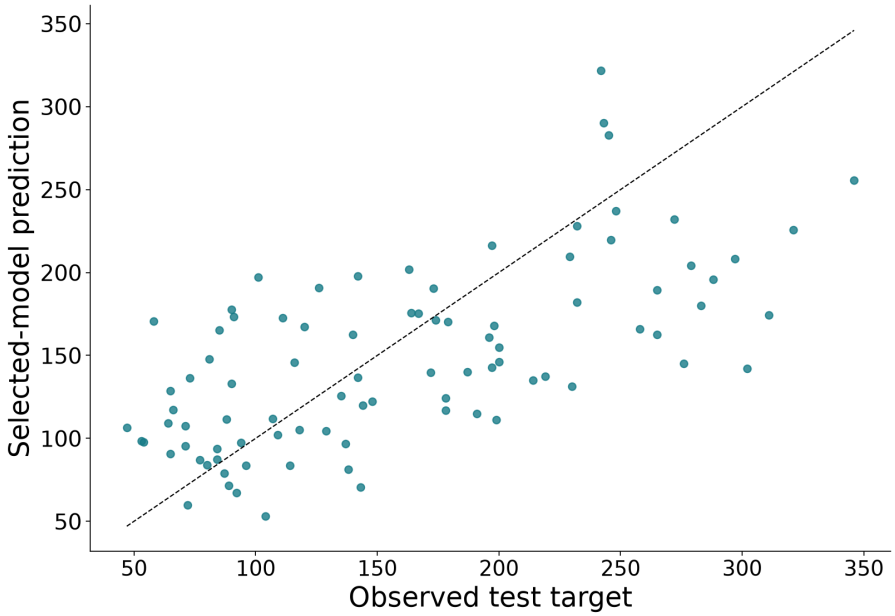


Figure 7.4: Actual selected-model final-test predictions. A later perfect computational replay does not remove these prediction errors against outcomes.

If the selected candidate were the constant median, this feature interpretation would not apply. The notebook then inspects the separately predeclared Ridge-20 candidate and records its name. Do not silently interpret a different candidate as the selected one.

7.16 What belongs in the handover

The payload has six required files:

File	Role
`frozen.csv`	Exact source IDs, features and target
`split.json`	Ordered training, validation and test IDs
`config.json`	Feature order, choices, selected candidate, versions
`replay.py`	Readable validation and fitting procedure
`reference-predictions.csv`	ID-aligned reference outcomes/predictions
`result.json`	Validation/test evidence and limitations

A separate `manifest.json` records the SHA-256 of each payload. It does not hash itself, which would be circular. The exported ZIP also retains actual plots and the fresh-process result.

7.16.1 Integrity is not authentication

SHA-256 detects changed bytes relative to a known reference digest. Parsing records checks a different identity. Appending a blank CSV line can preserve parsed rows while changing its hash.

An unsigned manifest does not authenticate an unknown sender: someone able to replace both payload and digest can create another internally consistent package. Neither hashes nor schemas validate scientific labels or a performance claim.

The lab avoids loading a model pickle by reconstructing from CSV, JSON and readable source. Pickle-based loaders may execute arbitrary code. Executing Python source is also consequential: review and trust the generated runner before running it. A checksum does not make arbitrary downloaded code safe.

7.17 Replay outside notebook memory

The notebook launches a new Python interpreter in a new temporary working directory, passing only an explicit handover path and output path. The runner first verifies the required-file set and hashes, then validates schema and split membership, then refits the frozen candidate and predicts the saved test IDs. Notebook globals cannot rescue missing files.

Check exact test-ID sequence before numerical comparison. For each value, the comparison uses

$$|p_i^{\text{replay}} - p_i^{\text{reference}}| \le a + r|p_i^{\text{reference}}|, \qquad a = 10^{-8},\ r = 10^{-10}.$$

For reference value 10, the allowed difference is 1.1×10^{-8}. This tolerance is a numerical-agreement rule, not a model-accuracy score. Inspect input bytes, IDs, feature order, configuration and versions before changing it.

The actual local reference maximum prediction difference is 0.0. That establishes a replay under the recorded installed environment, not all future software versions. NumPy's random-stream compatibility documentation imposes strict conditions involving seed, generator, call sequence, build, environment and machine; do not overstate a seed-only guarantee.

7.18 Test deliberate failures on copies

Corrupt only temporary copies of the data. Inject a duplicate ID, a non-finite value, or a reordered header. Catch the expected `ValueError` and record its message. An accepted malformed copy must trigger a failed assertion. Recheck the original data hash afterward.

7.19 Eight core exercises and two optional challenges

Each task requires a prediction before running, an actual observation, an explanation and a stopping check. Alternate driver and reviewer in pairs. Use H1 for Tasks 1–2, guided H2 for Tasks 3–5, and H3 for remaining work, review and export. These tasks share the 120 minutes of practice; they are not extra hours after the walkthrough.

1. **Row-order trap:** reverse a 20-ID toy dataset, reuse seeded positional indices, then recover frozen membership by ID. Explain set equality versus sequence equality.
2. **Preprocessing scope:** assert the fitted scaler mean equals training means. Contrast all-row means descriptively without fitting a leaky competing model or claiming a particular score change.

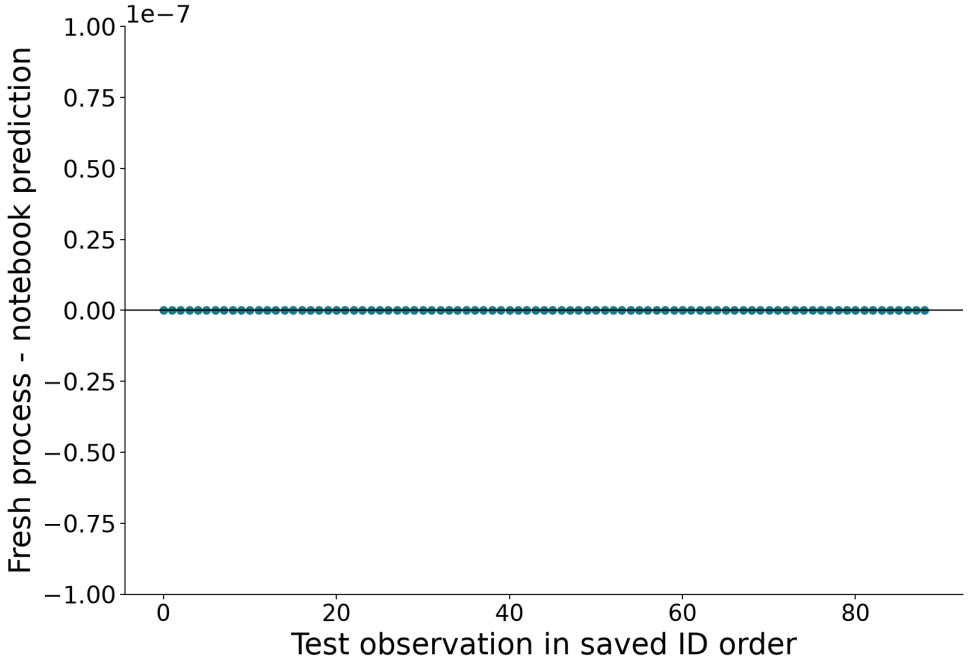


Figure 7.5: Actual fresh-process prediction differences after exact ID alignment. The vertical scale is explicit. Replay difference and residual against observed target answer different questions.

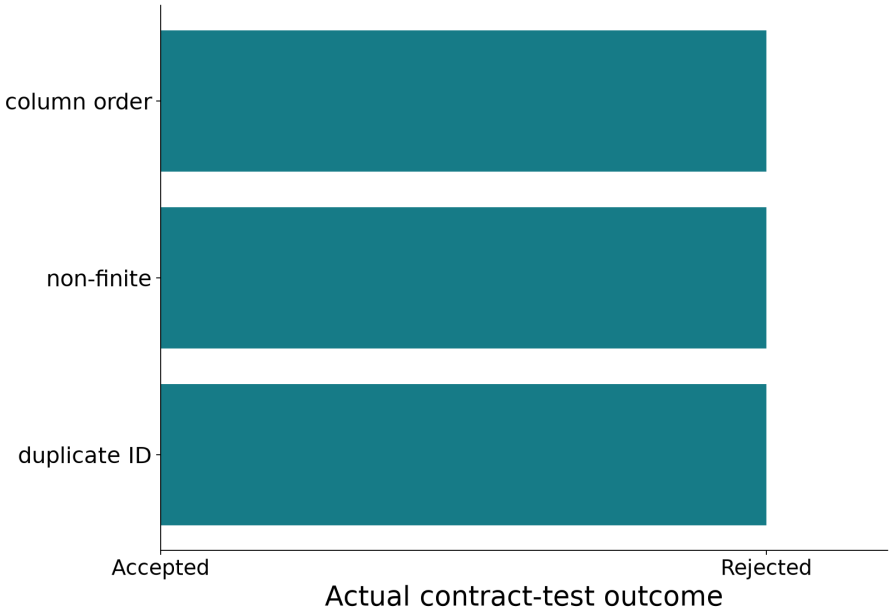


Figure 7.6: Actual three-case contract rejection test. These outcomes test an interface, not clinical safety or predictive accuracy.

3. **Selection rule:** recompute the validation-MAE winner and declared tie order. Never use final-test results to choose it.
4. **Contract failures:** reject duplicate IDs, NaN and reordered columns on temporary copies. Confirm the original CSV unchanged.
5. **Fresh process:** execute the reviewed runner in another temporary working directory. Compare saved ID sequences and predictions under the explicit tolerance.
6. **Controlled extension:** fit alpha 200 on the same training rows and report validation only. Because final-test results have been seen, label this exploratory; a new selection claim needs a fresh evaluation protocol. Preserve the original winner.
7. **Local explanation:** add 0.1 training SD to one declared feature and verify the fitted-function prediction change. Record the candidate and exclude causal or clinical claims.
8. **Handover audit:** verify every payload digest and identify one actual failure row. Write claim, evidence, limitation and next evaluation so a partner can locate the supporting files.

Optional A changes a copied CSV and confirms manifest rejection before training. Optional B explicitly restores reordered numeric columns by unique names and verifies exact recovery of the canonical matrix. Both preserve the reference package and neither is required for core completion.

7.19.1 Student responses and evidence export

The 19 core fields comprise eight predictions, eight explanations, controlled change, limitation and AI-use/checking disclosure (or "none"). Published responses and outputs start blank. Missing fields produce status `draft`; nonempty fields mean `ready for peer discussion`, not a correctness grade. Rerun only the final export after editing responses; initialization resets them.

Download both the handover ZIP and exercise ZIP before the free runtime ends. After code review, the extracted runner can be executed in a compatible environment using the explicit root and output paths shown in the notebook. No paid AI, GPU, API key or external dataset download is needed. AI may help explain code or suggest hypotheses, but it must not fabricate execution evidence or measured values.

7.20 Instructor execution and limits

On 19 October 2026 both notebooks were fully executed in fresh local CPU Jupyter kernels. Each invoked independent replay interpreters. All eight core tasks, both optional challenges, draft/completed export branches, 11 plot exports, numerical checks and ZIP integrity were checked. This is not a claim that the Google Colab browser itself was used for execution. Actual versions are recorded in the evidence JSON; a local software check does not establish clinical, causal or external deployment validity.

Reference CSV SHA-256:

`a589d3035403d59869ec18c1007e379a396c9cc53b562cb979be676f2b85c4d4`

7.21 Summary and examination bridge

Freeze identities, choices, transformations and environment. Reject malformed interfaces before fitting. Compare aligned prediction vectors in a new process, then state exactly what agreement supports. Keep predictive error, replay error, file integrity and scientific validity separate. For the written and oral midterm, be ready to explain one choice, one failure and one change that would invalidate the claim.

7.22 Primary-source reading

scikit-learn: raw dataset API

scikit-learn: preprocessing pitfalls

scikit-learn: persistence and security

NumPy: random-stream compatibility

7.23 Weeks 10–11: workflow and examination preparation

The workflow studio assembles a problem brief, data contract, train/validation/test rule, baseline, metric, error analysis and bounded recommendation. Existing designated Homework requirements remain separate from formative weekly exercises. The written examination checks conceptual understanding. The oral examination asks students to explain choices, interpret a failure case and identify one change that would invalidate a result. Practice notebooks do not replace the written or oral examination.

Checkpoint

A model has a strong overall score but fails near a sensor detection limit. State one evaluation slice, one data-collection action and one possible operating restriction. Name the additional evidence needed before recommending that restriction.

Part II

Language Models, Agents, and Engineering Workflows

Chapter 8

Transformers and Large Language Models

Week 12. This unit provides the minimum technical frame required to use LLMs in an engineering workflow without confusing fluent output with verified knowledge.

Learning outcomes

- Describe tokenization, next-token prediction, and attention at a high level.
- Explain why a language model can produce useful text without a guaranteed factual basis.
- Distinguish prompting, retrieval, fine-tuning, and tool use.
- Identify an engineering task for which an LLM should assist rather than decide.

8.1 Attention and next-token prediction

Transformers use attention to weight relationships among tokens in a context sequence [15]. Language-model training commonly predicts the next token. This objective can produce broadly useful representations, but it does not create a built-in source of current, task-specific, or verified information.

8.2 Use a bounded role

An LLM may summarize a lab note, propose code changes, draft a test plan, or translate a structured result into prose. The workflow must state what the model may access, which claims require retrieval or calculation, and how a human checks the result. Prompting is a form of interface design, not evidence.

Colab Lab

Week 12: use a small prompt experiment to compare an unconstrained request with a request that specifies context, output format, and verification step. Record failure modes rather than ranking models by style.

Chapter 9

Retrieval, Tools, and Structured Knowledge

Week 13. Retrieval and tools reduce the need for a language model to invent information. They also introduce new interfaces that must be tested: document selection, tool arguments, return values, permissions, and error handling.

Learning outcomes

- Describe the retrieve–ground–answer pattern.
- Separate a source document, retrieved passage, model response, and citation.
- Define a simple tool contract with inputs, outputs, and failure behaviour.
- Evaluate a retrieval workflow using task-relevant test questions.

9.1 Retrieval-augmented generation

A retrieval-augmented workflow first selects relevant material, then supplies it as context for a response [7]. The model can still misunderstand or ignore the material. Therefore the output should expose which source passages were used and decline to answer when the required evidence is absent.

9.2 Tools are software interfaces

A tool call has a schema, preconditions, outputs, and errors. Treat it as a software component: validate inputs, log execution, test representative failures, and keep high-impact actions behind human confirmation. The final project may use a mock tool when a real service would require paid access or confidential data.

Colab Lab

Week 13: build a small local retrieval example from a supplied text collection. Compare an answer with and without retrieved context, then inspect one case where retrieval is insufficient.

Chapter 10

Agent Design for Engineering Workflows

Week 14. An agent is not defined by a conversational interface. In this course, an agent is a bounded loop that observes a state, selects an allowed action, records evidence, and stops under explicit conditions.

Learning outcomes

- Write a goal, state representation, action set, and stopping condition.
- Decide when a workflow needs an agent loop rather than a single prompt or script.
- Define human approvals for irreversible or high-impact actions.
- Create test cases for expected and adverse agent behaviour.

10.1 A minimal agent loop

A useful abstraction is: observe → plan → act → verify → record. The loop should be narrow enough to test. Tool use, memory, and reflection are optional components, not automatic signs of quality. Reasoning-and-action patterns such as ReAct motivate explicit tool traces, but a trace is still evidence to be inspected, not proof of correctness [16].

10.2 Agent plan Homework

Homework 3 specifies a problem boundary, inputs, permitted tools, output format, stop condition, human approval point, and at least five failure cases. The plan must be feasible with a free or basic-credit tool path and must include a non-LLM fallback for a core calculation or validation step.

Checkpoint

A proposed "research agent" can search documents, calculate a summary statistic, and send an email. Which actions require independent tests? Which action requires human confirmation? Where should the trace be stored?

10.3 Current agent engineering patterns

Contemporary agent tooling increasingly separates the decision harness from the environment in which untrusted or model-generated code runs. The practical course translation is provider-neutral: expose a small tool contract, constrain the workspace, store a trace, checkpoint useful state, and require review before a high-impact action. These boundaries make a student prototype easier to test and recover when a run fails [10]. They do not make the agent correct; they make its behaviour more observable.

Chapter 11

Testing, Vibe Coding, and the Final Project

Weeks 15–18. Vibe coding is a fast way to express an intent to an AI coding tool. It does not transfer accountability. The course uses it as an implementation technique within a testable engineering workflow.

Learning outcomes

- Turn a natural-language change request into observable acceptance criteria.
- Review an AI-generated patch for data flow, assumptions, and failure behaviour.
- Design a small test suite before or alongside an agent implementation.
- Demonstrate a final workflow with evidence, limits, and a rollback path.

11.1 Vibe coding with boundaries

A responsible request names the intended user, input/output contract, constraints, and tests. After a tool proposes code, inspect the diff, run the relevant notebook cells or tests, and explain what changed. Avoid asking a model to make broad changes to an unexamined codebase. Prefer small, reversible changes with a visible before/after result.

11.2 Evaluation before demonstration

The final project needs a compact evaluation set that includes ordinary cases, edge cases, and known failure cases. For each case, record expected behaviour, actual behaviour, evidence, and follow-up action. A polished demo without a reproducible trace does not clear the project requirement.

11.3 Final assessment

The final project demonstration explains the problem boundary, agent or workflow design, evidence from tests, remaining limitations, and a next safe step. The written examination checks conceptual transfer. The oral examination can ask students to reason about a tool contract, an AI-generated patch, an evaluation trace, a rollback, or an unexpected failure.

Colab Lab

Weeks 15–17: use the project studio to build in small steps. Preserve the baseline, record every meaningful change, and rehearse the explanation of one failure case before the final demonstration.

11.4 Risk-aware evaluation

For the final project, evaluate more than apparent task completion. Include an ordinary case, an adverse or ambiguous case, a tool failure, and a case that must escalate to a human. Record the risk, expected behaviour, observed behaviour, evidence, and mitigation. This course-level practice is consistent with the lifecycle emphasis of the NIST Generative AI Profile, while remaining small enough for a student project [9].

Appendix A

Colab Conventions and Homework Milestones

A.1 Colab conventions

Every student notebook should begin with a short environment cell, use a declared random seed where relevant, and provide a CPU-compatible path. Optional accelerator cells must not be required for assessment. Notebooks should obtain public data from a documented source or a course repository path; they must not include credentials, personal data, or hidden instructor solutions.

The course website opens each notebook through a public GitHub repository using the Colab URL pattern defined by `\CourseRepo` in `main.tex`. Replace that one configuration value when the repository is created. The notebook source remains the canonical student artifact; the rendered website is an access layer.

A.2 Homework milestones

Homework	Timing	Evidence required
ML Problem Brief	Week 5	Decision context, target, features, split proposal, metric, and one risk.
ML Workflow	Week 10	Executable notebook, baseline, evaluation, error analysis, and bounded conclusion.
Final Agent Plan	Week 14	Goal, tools, stop condition, approval point, failure cases, and test plan.
Demo Readiness	Week 16	Reproducible run, evaluation cases, trace, limitation, and rollback or safe fallback.

A.3 Submission rule

An AI tool can assist with code, prose, or debugging, but the student remains responsible for the submitted claim. Work that uses AI assistance should identify the assistance when it materially shaped the artifact, then show the validation performed by the student.

A.4 Runtime-aware Lab design

Colab can load notebooks from GitHub, but its free resource limits, hardware availability, and runtime lifetime change over time. Runtimes are ephemeral, so a notebook must install or check any required package, fetch only documented public data, and write important outputs to a student-controlled location only when the student explicitly chooses to do so [3]. Core Labs in this course are CPU-first; any GPU cell is optional and cannot be necessary for a graded result.

The current reference implementation uses scikit-learn for core ML workflows and PyTorch only after neural-network foundations. Rather than pinning the course to a fragile hosted-runtime version, each notebook records the actual package versions at execution time. This makes a future result diagnosable when managed environments change [11, 13].

Appendix B

Extension Studios: Deep Learning Architectures

These topics remain available as optional Colab studios, reading groups, or final-project extensions. They are deliberately not prerequisites for the core ML workflow.

B.1 Convolutional neural networks

CNNs use shared local filters to exploit image structure. LeNet and later residual architectures illustrate how representation depth, skip connections, and data scale change a vision workflow [6]. Use this studio when the course data are images and a classical baseline is already in place.

B.2 Sequential models

RNNs, LSTMs, and GRUs model ordered observations. Their main lesson for this course is temporal data discipline: define what is known at each prediction time and avoid future leakage [5].

B.3 Generative models

GANs formulate generation as an adversarial game [2]; diffusion models learn a reverse denoising process [4]. These are useful extension topics for representation and synthesis, but they do not replace the need for an evaluation contract.

Bibliography

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Ian Goodfellow et al. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.

[3] Google Colaboratory. Frequently asked questions: Resource limits and notebook runtimes. https://research.google.com/colaboratory/faq.html, 2026. Accessed August 2026.

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.

[5] Sepp Hochreiter and Jurgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[6] Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[7] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[8] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[9] National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, NIST, 2024. URL https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf.

[10] OpenAI. The next evolution of the agents sdk. https://openai.com/index/the-next-evolution-of-the-agents-sdk/, 2026. Accessed August 2026.

[11] PyTorch Contributors. Pytorch stable documentation. https://docs.pytorch.org/docs/stable/, 2026. Accessed August 2026.

[12] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[13] scikit-learn developers. scikit-learn 1.9 release history. https://scikit-learn.org/stable/whats_new.html, 2026. Accessed August 2026.

[14] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, 1995.

[15] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[16] Shunyu Yao et al. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.